API Design and Specification

Comprehensive API design creates clear contracts between frontend and backend systems while following RESTful principles and industry standards. This section creates detailed API specifications using OpenAPI standards with proper authentication, validation, and error handling.

Exercise 1.1: OpenAPI Specification Design

Create comprehensive OpenAPI specifications for all chatbot endpoints:

~~~
Design detailed OpenAPI specification for our chatbot API:

Use the Web Search MCP server to research current OpenAPI best practices and RESTful design patterns, then create:

OpenAPI 3.0 Base Specification:
```yaml
openapi: 3.0.3
info:
  title: Chatbot API
  description: AI-powered chatbot API with authentication and conversation management
  version: 1.0.0
  contact:
    name: API Support
    email: api-support@chatbot.com
  license:
    name: MIT
    url: https://opensource.org/licenses/MIT

servers:
  - url: https://api.chatbot.com/v1
    description: Production server
  - url: https://staging-api.chatbot.com/v1
    description: Staging server
  - url: http://localhost:3000/v1
    description: Development server

security:
  - BearerAuth: []
  - ApiKeyAuth: []

components:
  securitySchemes:
    BearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
    ApiKeyAuth:
      type: apiKey
      in: header
      name: X-API-Key
```

Authentication Endpoints Specification:
```yaml
paths:
  /auth/register:
    post:
      summary: Register new user
      description: Create a new user account with email and password
      tags: [Authentication]
      security: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, password, firstName, lastName]
              properties:
                email:
                  type: string
                  format: email
                  example: user@example.com
                password:
                  type: string
                  minLength: 12
                  pattern: '^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]'
                  example: SecurePass123!
                firstName:
                  type: string
                  minLength: 1
                  maxLength: 50
                  example: John
                lastName:
                  type: string
                  minLength: 1
                  maxLength: 50
                  example: Doe
      responses:
        '201':
          description: User created successfully
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/AuthResponse'
        '400':
          $ref: '#/components/responses/BadRequest'
        '409':
          $ref: '#/components/responses/Conflict'
        '422':
          $ref: '#/components/responses/ValidationError'

  /auth/login:
    post:
      summary: Authenticate user
      description: Login with email and password to receive JWT token
      tags: [Authentication]
      security: []
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [email, password]
              properties:
                email:
                  type: string
                  format: email
                password:
                  type: string
                rememberMe:
                  type: boolean
                  default: false
      responses:
        '200':
          description: Authentication successful
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/AuthResponse'
        '401':
          $ref: '#/components/responses/Unauthorized'
        '422':
          $ref: '#/components/responses/ValidationError'
```

Chat Endpoints Specification:
```yaml
  /chat/conversations:
    get:
      summary: List user conversations
      description: Retrieve paginated list of user's conversations
      tags: [Chat]
      parameters:
        - name: page
          in: query
          schema:
            type: integer
            minimum: 1
            default: 1
        - name: limit
          in: query
          schema:
            type: integer
            minimum: 1
            maximum: 100
            default: 20
        - name: search
          in: query
          schema:
            type: string
            maxLength: 100
      responses:
        '200':
          description: Conversations retrieved successfully
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/ConversationList'

    post:
      summary: Create new conversation
      description: Start a new conversation thread
      tags: [Chat]
      requestBody:
        content:
          application/json:
            schema:
              type: object
              properties:
                title:
                  type: string
                  maxLength: 100
                  example: "Help with project planning"
      responses:
        '201':
          description: Conversation created successfully
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/Conversation'

  /chat/conversations/{conversationId}/messages:
    post:
      summary: Send message
      description: Send a message in a conversation and receive AI response
      tags: [Chat]
      parameters:
        - name: conversationId
          in: path
          required: true
          schema:
            type: string
            format: uuid
      requestBody:
        required: true
        content:
          application/json:
            schema:
              type: object
              required: [content]
              properties:
                content:
                  type: string
                  minLength: 1
                  maxLength: 4000
                  example: "How can I improve my project management skills?"
      responses:
        '200':
          description: Message sent and response received
          content:
            application/json:
              schema:
                $ref: '#/components/schemas/MessageResponse'
```

Research current OpenAPI specification best practices and RESTful API design patterns using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive OpenAPI 3.0 specification with all endpoints documented
- Detailed request/response schemas with validation rules
- Proper authentication and security scheme definitions
- Clear endpoint descriptions and examples
- Industry-standard RESTful API design patterns

Exercise 1.2: Data Models and Schema Design

Create comprehensive data models and validation schemas:

~~~
Design detailed data models and validation schemas for our chatbot API:

**Core Data Models**:
```yaml
components:
  schemas:
    User:
      type: object
      required: [id, email, firstName, lastName, createdAt]
      properties:
        id:
          type: string
          format: uuid
          example: 123e4567-e89b-12d3-a456-426614174000
        email:
          type: string
          format: email
          example: user@example.com
        firstName:
          type: string
          minLength: 1
          maxLength: 50
          example: John
        lastName:
          type: string
          minLength: 1
          maxLength: 50
          example: Doe
        avatar:
          type: string
          format: uri
          nullable: true
        preferences:
          $ref: '#/components/schemas/UserPreferences'
        createdAt:
          type: string
          format: date-time
        lastLoginAt:
          type: string
          format: date-time
          nullable: true

    UserPreferences:
      type: object
      properties:
        theme:
          type: string
          enum: [light, dark, system]
          default: system
        language:
          type: string
          pattern: '^[a-z]{2}(-[A-Z]{2})?$'
          default: en-US
        notifications:
          type: object
          properties:
            email:
              type: boolean
              default: true
            push:
              type: boolean
              default: false

    Conversation:
      type: object
      required: [id, userId, title, createdAt, updatedAt]
      properties:
        id:
          type: string
          format: uuid
        userId:
          type: string
          format: uuid
        title:
          type: string
          maxLength: 100
          example: "Project Planning Discussion"
        messageCount:
          type: integer
          minimum: 0
        lastMessageAt:
          type: string
          format: date-time
          nullable: true
        createdAt:
          type: string
          format: date-time
        updatedAt:
          type: string
          format: date-time
        isArchived:
          type: boolean
          default: false

    Message:
      type: object
      required: [id, conversationId, role, content, createdAt]
      properties:
        id:
          type: string
          format: uuid
        conversationId:
          type: string
          format: uuid
        role:
          type: string
          enum: [user, assistant, system]
        content:
          type: string
          minLength: 1
          maxLength: 4000
        metadata:
          type: object
          properties:
            model:
              type: string
              example: claude-3-haiku
            tokens:
              type: integer
              minimum: 0
            processingTime:
              type: number
              minimum: 0
        createdAt:
          type: string
          format: date-time
        editedAt:
          type: string
          format: date-time
          nullable: true
```

**Response Models**:
```yaml
    AuthResponse:
      type: object
      required: [user, accessToken, refreshToken, expiresIn]
      properties:
        user:
          $ref: '#/components/schemas/User'
        accessToken:
          type: string
          example: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
        refreshToken:
          type: string
          example: def50200e54b0b...
        expiresIn:
          type: integer
          example: 3600
        tokenType:
          type: string
          example: Bearer

    MessageResponse:
      type: object
      required: [userMessage, assistantMessage]
      properties:
        userMessage:
          $ref: '#/components/schemas/Message'
        assistantMessage:
          $ref: '#/components/schemas/Message'
        conversation:
          $ref: '#/components/schemas/Conversation'

    ConversationList:
      type: object
      required: [conversations, pagination]
      properties:
        conversations:
          type: array
          items:
            $ref: '#/components/schemas/Conversation'
        pagination:
          $ref: '#/components/schemas/PaginationMeta'

    PaginationMeta:
      type: object
      required: [page, limit, total, totalPages]
      properties:
        page:
          type: integer
          minimum: 1
        limit:
          type: integer
          minimum: 1
        total:
          type: integer
          minimum: 0
        totalPages:
          type: integer
          minimum: 0
        hasNext:
          type: boolean
        hasPrevious:
          type: boolean
```

Research current data modeling best practices and validation patterns using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive data models with proper validation rules
- Consistent schema design patterns across all entities
- Proper use of OpenAPI data types and formats
- Reusable schema components for common patterns
- Clear relationships between different data models

Exercise 1.3: Error Handling and Status Codes

Design comprehensive error handling and HTTP status code usage:

~~~
Create detailed error handling specifications for our chatbot API:

**Error Response Models**:
```yaml
components:
  responses:
    BadRequest:
      description: Bad request - invalid input parameters
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
          example:
            error:
              code: BAD_REQUEST
              message: Invalid request parameters
              details: The request contains invalid or missing parameters
              timestamp: "2024-01-15T10:30:00Z"
              path: "/api/v1/chat/messages"

    Unauthorized:
      description: Unauthorized - authentication required
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
          example:
            error:
              code: UNAUTHORIZED
              message: Authentication required
              details: Valid JWT token must be provided in Authorization header

    Forbidden:
      description: Forbidden - insufficient permissions
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'

    NotFound:
      description: Resource not found
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'

    Conflict:
      description: Conflict - resource already exists
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'

    ValidationError:
      description: Validation error - invalid input data
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ValidationErrorResponse'

    RateLimitExceeded:
      description: Rate limit exceeded
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'
      headers:
        X-RateLimit-Limit:
          schema:
            type: integer
          description: Request limit per time window
        X-RateLimit-Remaining:
          schema:
            type: integer
          description: Remaining requests in current window
        X-RateLimit-Reset:
          schema:
            type: integer
          description: Time when rate limit resets (Unix timestamp)

    InternalServerError:
      description: Internal server error
      content:
        application/json:
          schema:
            $ref: '#/components/schemas/ErrorResponse'

  schemas:
    ErrorResponse:
      type: object
      required: [error]
      properties:
        error:
          type: object
          required: [code, message, timestamp]
          properties:
            code:
              type: string
              enum: [
                BAD_REQUEST,
                UNAUTHORIZED,
                FORBIDDEN,
                NOT_FOUND,
                CONFLICT,
                RATE_LIMIT_EXCEEDED,
                INTERNAL_SERVER_ERROR,
                SERVICE_UNAVAILABLE
              ]
            message:
              type: string
              description: Human-readable error message
            details:
              type: string
              description: Additional error details
            timestamp:
              type: string
              format: date-time
            path:
              type: string
              description: API endpoint that generated the error
            requestId:
              type: string
              format: uuid
              description: Unique request identifier for tracking

    ValidationErrorResponse:
      type: object
      required: [error]
      properties:
        error:
          type: object
          required: [code, message, timestamp, validationErrors]
          properties:
            code:
              type: string
              enum: [VALIDATION_ERROR]
            message:
              type: string
              example: Validation failed
            timestamp:
              type: string
              format: date-time
            path:
              type: string
            requestId:
              type: string
              format: uuid
            validationErrors:
              type: array
              items:
                type: object
                required: [field, message]
                properties:
                  field:
                    type: string
                    example: email
                  message:
                    type: string
                    example: Invalid email format
                  code:
                    type: string
                    example: INVALID_FORMAT
                  value:
                    description: The invalid value (optional)
```

**HTTP Status Code Usage Guidelines**:
```yaml
# Status Code Usage Patterns
statusCodeGuidelines:
  success:
    200: "OK - Successful GET, PUT, PATCH requests"
    201: "Created - Successful POST requests that create resources"
    204: "No Content - Successful DELETE requests"

  clientError:
    400: "Bad Request - Invalid request syntax or parameters"
    401: "Unauthorized - Authentication required or invalid"
    403: "Forbidden - Valid authentication but insufficient permissions"
    404: "Not Found - Resource does not exist"
    409: "Conflict - Resource already exists or state conflict"
    422: "Unprocessable Entity - Valid syntax but semantic errors"
    429: "Too Many Requests - Rate limit exceeded"

  serverError:
    500: "Internal Server Error - Unexpected server error"
    502: "Bad Gateway - Invalid response from upstream server"
    503: "Service Unavailable - Server temporarily unavailable"
    504: "Gateway Timeout - Upstream server timeout"
```

Research current API error handling best practices and HTTP status code conventions using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive error response models with consistent structure
- Proper HTTP status code usage following REST conventions
- Detailed validation error responses with field-level feedback
- Rate limiting headers and error responses
- Unique request IDs for error tracking and debugging

Exercise 1.4: API Versioning and Documentation

Create API versioning strategy and comprehensive documentation:

~~~
Design API versioning strategy and documentation standards:

**API Versioning Strategy**:
```yaml
# Versioning Approach
versioningStrategy:
  method: "URL Path Versioning"
  pattern: "/api/v{major}/endpoint"
  examples:
    - "/api/v1/chat/messages"
    - "/api/v2/chat/messages"

  versioningRules:
    majorVersion:
      increment: "Breaking changes that affect existing clients"
      examples:
        - "Removing endpoints or fields"
        - "Changing response structure"
        - "Modifying authentication requirements"

    minorVersion:
      increment: "Backward-compatible additions"
      examples:
        - "Adding new endpoints"
        - "Adding optional fields"
        - "Adding new optional parameters"

    patchVersion:
      increment: "Bug fixes and internal improvements"
      examples:
        - "Performance improvements"
        - "Bug fixes"
        - "Documentation updates"

# Version Lifecycle Management
versionLifecycle:
  support:
    current: "Full support with new features"
    previous: "Maintenance mode - bug fixes only"
    deprecated: "6 months notice before removal"
    sunset: "Version no longer available"

  deprecationProcess:
    announcement: "6 months advance notice"
    headers: "Sunset and Deprecation headers in responses"
    documentation: "Clear migration guides"
    support: "Developer support during transition"
```

**API Documentation Standards**:
```yaml
# Documentation Requirements
documentationStandards:
  endpoint:
    summary: "Brief description of endpoint purpose"
    description: "Detailed explanation of functionality"
    parameters: "All parameters with examples and constraints"
    requestBody: "Complete request schema with examples"
    responses: "All possible responses with examples"
    examples: "Multiple realistic usage examples"

  authentication:
    schemes: "All supported authentication methods"
    scopes: "Required permissions for each endpoint"
    examples: "How to authenticate requests"

  errorHandling:
    codes: "All possible error codes and meanings"
    examples: "Example error responses"
    troubleshooting: "Common issues and solutions"

  rateLimit:
    limits: "Rate limiting rules and headers"
    handling: "How to handle rate limit responses"
    best practices: "Recommendations for efficient API usage"

# Interactive Documentation
interactiveDocumentation:
  swagger-ui:
    features:
      - "Try it out functionality"
      - "Request/response examples"
      - "Authentication testing"
      - "Schema validation"

  postman:
    collections: "Complete Postman collection with examples"
    environments: "Development, staging, production environments"
    tests: "Pre-written tests for common scenarios"

  sdks:
    javascript: "TypeScript SDK with full type definitions"
    python: "Python SDK with async support"
    curl: "cURL examples for all endpoints"
```

**API Testing and Validation**:
```yaml
# Contract Testing
contractTesting:
  openapi-validator:
    purpose: "Validate API responses against OpenAPI spec"
    integration: "Automated testing in CI/CD pipeline"
    coverage: "All endpoints and response codes"

  pact-testing:
    purpose: "Consumer-driven contract testing"
    implementation: "Frontend-backend contract validation"
    automation: "Automated contract verification"

  schema-validation:
    request: "Validate all incoming requests"
    response: "Validate all outgoing responses"
    errors: "Proper error responses for invalid data"

# API Monitoring
apiMonitoring:
  health-checks:
    endpoint: "/health"
    checks: ["Database connectivity", "External service availability"]
    format: "Standard health check response format"

  metrics:
    performance: "Response times and throughput"
    errors: "Error rates by endpoint and status code"
    usage: "API usage patterns and trends"

  alerting:
    thresholds: "Performance and error rate thresholds"
    notifications: "Real-time alerts for API issues"
    escalation: "Escalation procedures for critical issues"
```

Set up Agent Hooks to automatically validate OpenAPI specifications when API code is modified and generate updated documentation.

Research current API documentation and versioning best practices using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive API versioning strategy with clear lifecycle management
- Detailed documentation standards ensuring consistency and completeness
- Interactive documentation with testing capabilities
- Contract testing and validation processes
- API monitoring and health check implementations

Key Deliverables

After completing the API Design and Specification exercises, you should have:

1. **OpenAPI Specification**: Complete API documentation with all endpoints and schemas
2. **Data Models**: Comprehensive data models with validation rules and relationships
3. **Error Handling**: Consistent error responses and HTTP status code usage
4. **Versioning Strategy**: API versioning approach with lifecycle management
5. **Documentation Standards**: Interactive documentation and testing capabilities

Integration with Advanced Features

- **Agent Hooks**: For automated API validation and documentation generation

Next Steps

With comprehensive API specifications established, you're ready to implement the business logic that powers your chatbot application. The next section will focus on service layer architecture, domain modeling, business rules implementation, and clean separation of concerns that bring your API specifications to life.

Previous Next

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Privacy policy Terms of use Cookie preferences